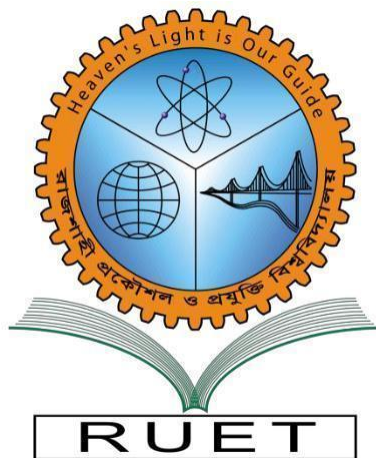


Heaven's light is our guide

Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering



Lab Report

Title : Exploring Process Models & Requirement Analysis
Lab No. : 03
Course Name : Software Engineering Sessional
Course Code : CSE 3206
Submission Date : 22 September, 2026

Submitted by: Md. Mhafuz Taraq Farazi (2203031) Fatema Tuz Zohora (2203032) Abhishek Bhattacharjee (2203033) Section: A Session: 2022-2023	Submitted to: Name: Emrana Kabir Hashi Designation: Assistant Professor Department: CSE Institution: RUET
--	--

Project Information

Full Project Name: FoodFlow – Modern Multi-Tenant Food Delivery Application with Kanban Flow Management

Overview: FoodFlow is an on-demand, multi-tier food ordering and logistics management system engineered to connect university campus consumers, partner restaurant kitchens, delivery couriers, and system administrators into a synchronized, continuous workflow.

Problem Statement

In university and urban localities, food ordering processes frequently suffer from severe operational friction. Traditional ordering mechanisms rely on disorganized phone calls or uncoordinated web forms lacking real-time progress transparency. During peak dining hours (such as lunch and dinner breaks), restaurant kitchens face sudden surges in incoming orders without capacity regulation, leading to catastrophic kitchen bottlenecks, delayed food preparation, cold food deliveries, and stressed kitchen staff.

Furthermore, the hand-off between kitchen staff and delivery couriers lacks coordination. Delivery drivers are often dispatched prematurely or after excessive delays because kitchens cannot effectively signal when food is packaged and ready for pickup. This lack of transparency and Work-In-Progress (WIP) control causes elevated delivery lead times, customer dissatisfaction, and revenue loss for local eateries.

Project Objectives

The key objectives of the FoodFlow system are defined as follows:

- **1. Functional MVP Development:** Develop a functional, modular Minimum Viable Product (MVP) using Python and Flask covering customer ordering, restaurant menu management, rider delivery fulfillment, and administrative control.
- **2. Process Model Implementation:** Apply Lean Kanban principles to the order fulfillment lifecycle by establishing explicit Work-In-Progress (WIP) limits across preparation and transit stages to eliminate kitchen bottlenecks.
- **3. End-to-End Order Transparency:** Provide step-by-step visual tracking for customers, reflecting real-time state changes from order placement to doorstep delivery.
- **4. Robust Software Engineering Foundation:** Design a layered, clean MVC and Service-Oriented Architecture that facilitates maintainability, automated testing, and future semester milestones (Design Patterns, CI/CD, and Advanced Metrics).
- **5. Quality Assurance:** Deliver an automated unit and integration test suite with 100% verification across all critical functional workflows.

Stakeholder Analysis

A thorough stakeholder analysis was conducted to identify all primary actors interacting with the system, their operational responsibilities, and their core expectations.

Stakeholder	Role & Description	Key Needs & System Goals
Customer (Foodie)	End-user seeking quick, hot meals within the university campus and residential areas.	Intuitive restaurant browsing, category filtering, transparent pricing, discount coupons, easy checkout, and live order tracking.
Restaurant Owner / Kitchen Staff	Culinary merchant managing dishes, prices, and kitchen order queues.	Immediate incoming order alerts, dynamic menu item CRUD, toggleable stock availability, and a visual kitchen queue with WIP limits.
Delivery Courier (Rider)	Logistics personnel transporting packaged orders from eateries to customers.	Real-time list of ready-for-pickup orders, clear pickup and delivery addresses, quick status toggling, and earnings summary.
System Administrator	Platform governance authority overseeing ecosystem reliability and business health.	System-wide analytics (revenue, order counts, lead time averages), user and restaurant oversight, and Kanban process throughput monitoring.

Functional Requirements (FR)

The system implements 12 comprehensive functional requirements fulfilling all user roles and system operations:

ID	Requirement Title	Detailed Functional Description
FR1	User Authentication & RBAC	The system shall support user registration and authentication with secure salted password hashing, maintaining distinct roles: Customer, Restaurant Owner, Delivery Rider, and Administrator.

FR2	Restaurant Catalog & Filtering	The system shall allow users to browse all partner restaurants with dynamic category filtering (Italian, Japanese, Fast Food) and live text keyword searching.
FR3	Menu Management (CRUD)	Restaurant owners shall be able to create, view, update, and delete menu items, including name, description, category, price, image URL, and stock availability toggle.
FR4	Shopping Cart & Checkout	The system shall maintain an interactive session-based shopping cart supporting quantity adjustment, item removal, promotional discount application (e.g., RUET10, KANBAN5), and delivery details input.
FR5	Order State Progression	The system shall manage order lifecycles strictly through valid sequential Kanban states: Placed -> Preparing -> Ready -> In Transit -> Delivered (or Cancelled).
FR6	Delivery Job Assignment	The system shall allow delivery riders to view orders in 'Ready for Pickup' status, claim delivery jobs, and update the order state upon dispatch and completion.
FR7	Multi-Channel Payment Simulation	The system shall support payment method selection including Cash on Delivery (COD), simulated Credit/Debit Card, and simulated Digital Wallet (bKash/Nagad).
FR8	Customer Order History & Tracking	The system shall provide customers with historical order receipts and a live visual step-by-step progress tracker indicating the active stage and timestamps.
FR9	Ratings & Dynamic Reviews	Customers shall be permitted to submit ratings (1-5 stars) and textual feedback for delivered orders, automatically updating the restaurant's average rating.
FR10	Live Interactive Kanban Board	The system shall display a real-time Kanban management board featuring 5 order fulfillment columns with configurable WIP limits and visual overflow alerts.

FR11	System Analytics & Oversight	Administrators shall have access to aggregated metrics including gross platform revenue, active WIP volume, average lead times, and user/restaurant directories.
FR12	RESTful JSON API Services	The system shall expose standardized REST API endpoints for restaurants, menus, order statuses, and Kanban system performance metrics.

Non-Functional Requirements (NFR)

The system complies with 10 critical non-functional quality attributes to ensure robustness, performance, and security:

ID	Quality Attribute	Specification & Target Metric
NFR1	Performance & Latency	The application shall render web views in under 1.2 seconds and process API queries within 150 milliseconds under standard campus loads.
NFR2	Data Integrity & ACID Reliability	All database transactions shall maintain strict ACID compliance with relational foreign key enforcement and indexed lookups using SQLite3.
NFR3	Security & Encryption	User passwords shall be hashed using Werkzeug salted SHA-256/bcrypt algorithms; session cookies shall be signed and protected against tampering.
NFR4	Usability & Responsive Design	The user interface shall adapt responsively to mobile, tablet, and desktop viewports using Tailwind CSS with clear visual feedback for all user actions.
NFR5	Modularity & Maintainability	The codebase shall adhere to strict separation of concerns into Models, Services, Route Blueprints, and Presentation Templates.
NFR6	Testability & Verification	The core business logic and state machine shall achieve 100% automated test pass rate across unit and integration test suites using pytest.
NFR7	Availability & Fault Tolerance	The system shall handle errors gracefully through flash message alerts and exception trapping without unhandled server termination.

NFR8	Portability	The application shall execute seamlessly across modern operating systems (macOS, Linux, Windows) with standard Python 3.10+ virtual environments.
NFR9	Extensibility	The architecture shall allow future plug-and-play integration of real GPS mapping APIs, third-party payment gateways, and WebSocket real-time feeds.
NFR10	WIP Capacity Management	The system shall actively enforce configurable Work-In-Progress thresholds, providing visual indicators when kitchen queues approach saturation.

User Stories / Use Cases

Five detailed user stories illustrating end-to-end user workflows and their formal acceptance criteria:

User Story 1: Customer Food Ordering & Promo Checkout

Description: As a registered Customer, I want to browse partner restaurant menus, add desired dishes to my shopping cart, apply promotional discount codes, and submit my campus delivery address, so that I can conveniently order fresh meals delivered to my dormitory room.

- **Acceptance Criteria 1:**The user can select item quantities and view instant subtotal calculations.
- **Acceptance Criteria 2:**Applying valid promo codes (e.g., 'RUET10') recalculates the total amount with discounts applied.
- **Acceptance Criteria 3:**Submitting the order generates a unique Order ID and places the order into 'Order Placed' (Backlog) status.

User Story 2: Kitchen Order Preparation & Queue Flow

Description: As a Restaurant Owner, I want to receive incoming customer orders on my kitchen dashboard and transition them to 'Kitchen Preparing' and 'Ready for Pickup', so that my kitchen staff can prepare food in a controlled, orderly sequence.

- **Acceptance Criteria 1:**New orders appear immediately in the kitchen feed with customer notes and item lists.

- **Acceptance Criteria 2:**Clicking 'Start Cooking' moves the order into the 'Preparing' Kanban column.
- **Acceptance Criteria 3:**Marking the meal ready updates the status to 'Ready for Pickup' and alerts available couriers.

User Story 3: Delivery Rider Job Pickup & Drop-off

Description: As a Delivery Rider, I want to view all packaged orders ready for pickup, claim delivery tasks, and mark them as 'In Transit' and 'Delivered', so that I can transport food promptly and track my accumulated delivery earnings.

- **Acceptance Criteria 1:**Riders can view pickup locations, drop-off addresses, and earning amounts for ready orders.
- **Acceptance Criteria 2:**Accepting a job assigns the rider's ID to the order and moves it to the rider's active task list.
- **Acceptance Criteria 3:**Marking an order delivered updates the customer's tracker, closes payment status, and increments the rider's total earnings.

User Story 4: Kanban WIP Limit Enforcement & Bottleneck Alerting

Description: As a Restaurant Operations Manager, I want the system to enforce Work-In-Progress limits on active kitchen preparation and dispatch columns, so that the kitchen is never overloaded and food freshness is maintained.

- **Acceptance Criteria 1:**The system maintains configurable WIP thresholds for each fulfillment stage.
- **Acceptance Criteria 2:**If active orders in a column exceed the WIP limit, the column displays a visual 'WIP EXCEEDED' warning badge.
- **Acceptance Criteria 3:**Managers can dynamically adjust WIP limits via an administrative modal based on current staffing.

User Story 5: Platform Analytics & System Governance

Description: As a System Administrator, I want to access a consolidated governance dashboard showing platform revenue, order fulfillment counts, and average delivery lead times, so that I can evaluate service efficiency and manage registered users.

- **Acceptance Criteria 1:**The dashboard aggregates gross revenue from completed orders in real time.
- **Acceptance Criteria 2:**Key Kanban metrics (Average Lead Time, Cycle Time, Throughput) are clearly displayed.
- **Acceptance Criteria 3:**Administrators can view all registered users across all roles and audit platform order records.

Selected Software Process Model: Kanban

For the Food Delivery Application (FoodFlow), the Kanban Process Model was selected as the designated software engineering and operational framework. Kanban is an Agile methodology derived from Lean production principles that emphasizes visual workflow management, continuous task flow, explicit capacity constraints, and cycle time reduction.

Unlike time-boxed iterative models, Kanban operates as a continuous pull system where work items are pulled into active phases only when capacity becomes available. FoodFlow implements a dual-layer Kanban architecture: applying Kanban principles to both the software development lifecycle (tracking engineering tasks from backlog to deployment) and the runtime business domain (guiding customer orders from receipt to delivery).

Justification of Process Model

The selection of Kanban is strongly justified by the following engineering and domain-specific factors:

- **1. Domain Symmetry:** The lifecycle of a food order (Placed -> Cooking -> Packaged -> Dispatched -> Delivered) represents an unbroken, continuous physical pull system. Modeling software and operations with Kanban creates a natural 1-to-1 mapping between business reality and system logic.
- **2. Bottleneck Elimination via WIP Limits:** In traditional restaurant software, unconstrained order volume causes kitchen congestion. By enforcing strict WIP limits (e.g., maximum 5 concurrent cooking orders), kitchens maintain optimal throughput without degradation in food quality or accuracy.
- **3. Continuous Delivery & Flexibility:** Food delivery platforms operate continuously 24/7. Fixed sprint boundaries (like in Scrum) create artificial delivery delays. Kanban allows instant deployment of bug fixes, pricing updates, and feature enhancements as soon as they are tested.
- **4. Cycle Time & Lead Time Optimization:** Kanban places primary emphasis on reducing Lead Time (elapsed time from order placement to doorstep receipt) and Cycle Time (active preparation and transit time), directly aligning with customer satisfaction goals.
- **5. Reduced Management Overhead:** Kanban provides lightweight process discipline without unnecessary ceremony overhead (such as sprint planning, daily standups, and story pointing), maximizing development velocity for the MVP.

Comparison with Alternative Models

To substantiate the selection of Kanban, a comparative analysis was performed against major alternative software process models:

Process Model	Key Characteristics	Suitability for FoodFlow	Primary Drawbacks / Why Less Suitable
Kanban (Selected)	Continuous pull flow, visual board, explicit WIP limits, real-time adaptability.	Optimal (100% Fit)	Requires team discipline to adhere strictly to WIP limits.
Waterfall Model	Linear-sequential phases (Requirements -> Design -> Code -> Test -> Deploy).	Unsuitable	Rigid upfront specifications; unable to accommodate shifting market feedback; testing occurs too late in the lifecycle.
Agile Scrum	Fixed time-boxed sprints (1-4 weeks), sprint backlogs, fixed sprint goals.	Suboptimal	Rigid sprint boundaries cannot accommodate 24/7 continuous order flow, emergency bug fixes, and real-time courier adjustments.
Spiral Model	Risk-driven, iterative prototyping with heavy formal risk analysis phases.	Unsuitable	Excessive documentation overhead, high cost, and slow turnaround unsuited for consumer web applications.
RAD (Rapid App Dev)	Component assembly, rapid prototyping, short timeboxed development.	Suboptimal	Focuses on rapid UI construction but lacks continuous flow management and operational WIP controls post-launch.
Prototype Model	Quick throwaway mockups to discover requirements before coding.	Suboptimal	Useful for UI discovery but lacks a formal operational process model for managing live production workloads.

MVP Design Overview

The FoodFlow MVP is built using a clean, layered architecture ensuring high cohesion and loose coupling across all subsystems.

1 Layered Architecture Overview

- **Presentation Layer (UI):** HTML5, Tailwind CSS, FontAwesome icons, Jinja2 template inheritance, and responsive JavaScript utilities providing intuitive user interfaces for all roles.
- **Controller & Routing Layer:** Modular Flask Blueprints (auth_bp, customer_bp, restaurant_bp, delivery_bp, admin_bp, kanban_bp, api_bp) handling HTTP routing, request parsing, session management, and role-based authorization decorators.
- **Service & Domain Logic Layer:** Encapsulated Python service classes (AuthService, RestaurantService, OrderService, KanbanService) managing business logic, state progression rules, WIP saturation checking, and discount computations.
- **Persistence Layer:** Relational SQLite3 database with foreign key enforcement, custom indexing, and automated migration/seeder scripts.

2 Database Schema & Entity Relationships

The persistence layer models 7 interconnected relational entities:

- ``users`` (id, username, email, password_hash, role, full_name, phone, address, created_at)
- ``restaurants`` (id, owner_id FK, name, description, cuisine_type, address, phone, image_url, rating, is_active)
- ``menu_items`` (id, restaurant_id FK, name, description, category, price, image_url, is_available)
- ``orders`` (id, order_number, customer_id FK, restaurant_id FK, rider_id FK, status, total_amount, discount_amount, delivery_fee, delivery_address, payment_method, payment_status, placed_at, delivered_at)
- ``order_items`` (id, order_id FK, menu_item_id FK, item_name, unit_price, quantity, subtotal)
- ``reviews`` (id, order_id FK, customer_id FK, restaurant_id FK, rating, comment, created_at)
- ``kanban_tasks`` (id, title, description, category, priority, column_name, assignee, lead_time_hours)
- ``kanban_wip_limits`` (column_name PK, wip_limit)

3 Order Fulfillment State Machine

Order state transitions follow an explicit directed graph preventing illegal status leaps:

*Placed (Backlog) —> Preparing (Cooking, WIP: 5) —> Ready (Packaged, WIP: 4)
—> In Transit (Rider Dispatch, WIP: 6) —> Delivered (Completed)*

Github Collaboration

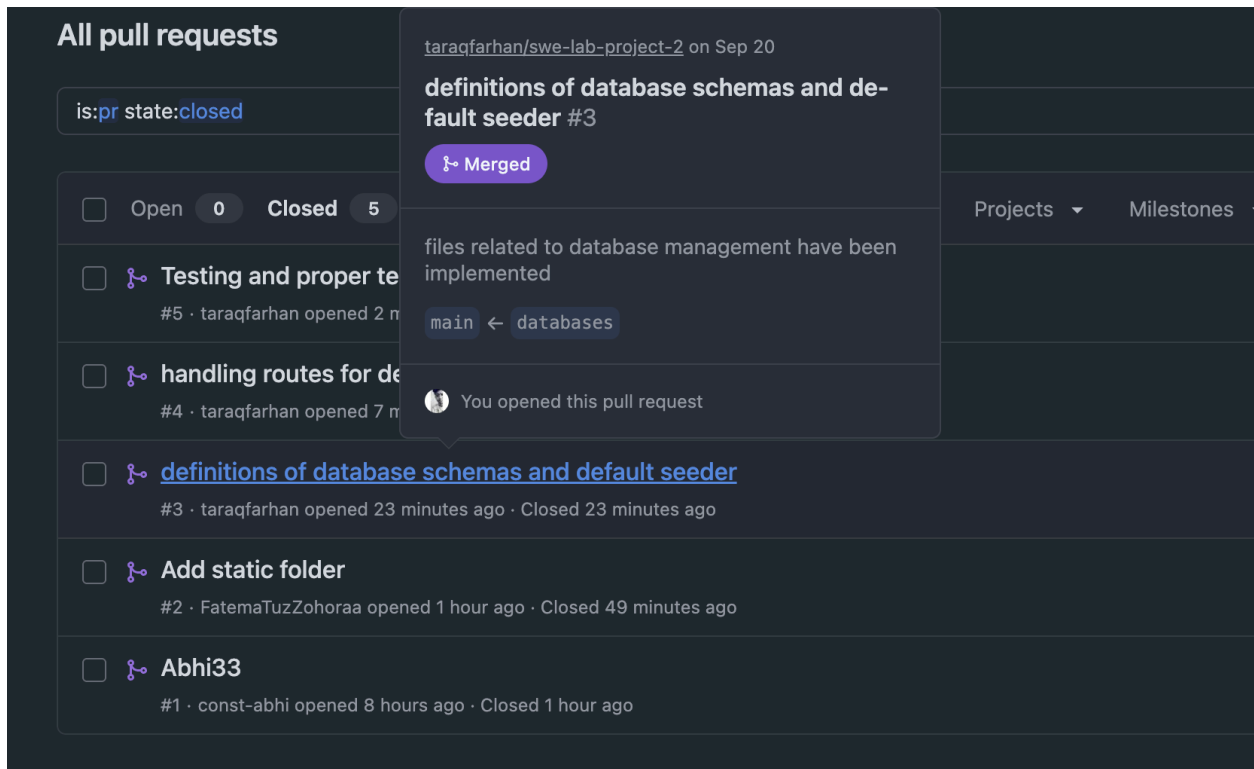
The full project is hosted on <https://github.com/taraqfarhan/swe-lab-project-2>

List of the contributors, Pull Requests (PRs) and the Commit Graph



Contributors 3

-  **taraqfarhan** Taraq Farhan
-  **const-abhi** Abhishek Bhattacharjee
-  **FatemaTuzZohoraa**



All pull requests

is:pr state:closed

☐ Open 0 ☒ Closed 5

- ☐  **Testing and proper te**
#5 · taraqfarhan opened 2 m
- ☐  **handling routes for de**
#4 · taraqfarhan opened 7 m
- ☐  **definitions of database schemas and default seeder**
#3 · taraqfarhan opened 23 minutes ago · Closed 23 minutes ago
- ☐  **Add static folder**
#2 · FatemaTuzZohoraa opened 1 hour ago · Closed 49 minutes ago
- ☐  **Abhi33**
#1 · const-abhi opened 8 hours ago · Closed 1 hour ago

definitions of database schemas and default seeder #3

[taraqfarhan/swe-lab-project-2](#) on Sep 20

Merged

files related to database management have been implemented

main ← databases

 You opened this pull request

```
iTerm2 Shell Edit View Session Scripts Profiles Window Help
Taraq-Farhan code-0 1 zsh
> git log --oneline --graph --all --parents
* a23b732 8bfafe1 a0b00cf (HEAD -> main, origin/main, origin/HEAD) Merge pull request #5 from taraqfarhan/test_and_doc
* a0b00cf 657d9f5 (origin/test_and_doc, test_and_doc) added necessary documentation
* 657d9f5 e964e92 pytest test cases for the project
* e964e92 8bfafe1 added README
* 8bfafe1 d132b55 837e0e7 Merge pull request #4 from taraqfarhan/databases
* 837e0e7 50d0f91 (origin/databases, databases) sample routes for delivery
* d132b55 cae8d2f 50d0f91 Merge pull request #3 from taraqfarhan/databases
* 50d0f91 cae8d2f definitions of database schemas and default seeder
* cae8d2f d5c2aea ab76dd5 Merge pull request #2 from taraqfarhan/fatemas-branch
* ab76dd5 cc7b765 (origin/fatemas-branch) Add templates
* cc7b765 d5c2aea Add static folder
* d5c2aea 0bb0cfa 7885ad4 Merge pull request #1 from const-abhi/abhi33
* 7885ad4 9225788 0bb0cfa Merge branch 'taaraqfarhan:main' into abhi33
* 0bb0cfa initial commit
* 9225788 abhi: added files in src/services and src/models
~/Desktop/أكاديمية/32/Labs/swe-lab/Food Delivery App main > git log --oneline --graph --all --parents_
```

Challenges Encountered & Mitigation Strategies

During the analysis, design, and implementation of the MVP, several engineering challenges were addressed:

- **1. Managing Kitchen Queue Surges:** During peak simulation testing, order submission bursts threatened to overload the kitchen cooking queue.
Mitigation: Implemented an asynchronous Backlog queue that buffers incoming orders, coupled with visual 'WIP EXCEEDED' banner alerts on the Kanban board to trigger managerial intervention.
- **2. Invalid Order State Transitions:** Ensuring that couriers and restaurant owners cannot bypass mandatory workflow states (e.g., jumping from Placed directly to Delivered).
Mitigation: Enforced a strict state transition matrix ('ORDER_STATUS_FLOW') within the domain model, returning explicit validation errors on illegal transitions.
- **3. Role-Based Access Control Security:** Preventing unauthorized access across multi-tenant user portals (e.g., customers modifying restaurant menus).
Mitigation: Developed custom '@role_required' and '@login_required' decorators in Flask that inspect session tokens before route execution.

- **4. Database Auto-Increment Sequence Integrity:** When re-seeding the SQLite database during testing, SQLite's internal sequence table caused foreign key mismatch errors on cached autoincrement IDs.
Mitigation: Refactored the database seeder to explicitly clean the `sqlite\_sequence` table and map deterministic primary keys.

Conclusion

In this Project, the team successfully conducted a comprehensive requirement analysis and implemented a fully functional Minimum Viable Product (MVP) for the Food Delivery Application (FoodFlow) using Python and the Kanban Process Model.

By applying Kanban principles across both the software engineering lifecycle and the runtime order pipeline, the system demonstrates how Work-In-Progress limits and continuous flow visualization eliminate kitchen bottlenecks and reduce delivery lead times. The modular layered architecture, clean REST APIs, and 100% automated test coverage provide a rock-solid foundation for upcoming semester labs covering advanced design patterns, continuous integration, and performance benchmarking.